

DAY 11

Bash Command Processing – Complete Detailed Notes

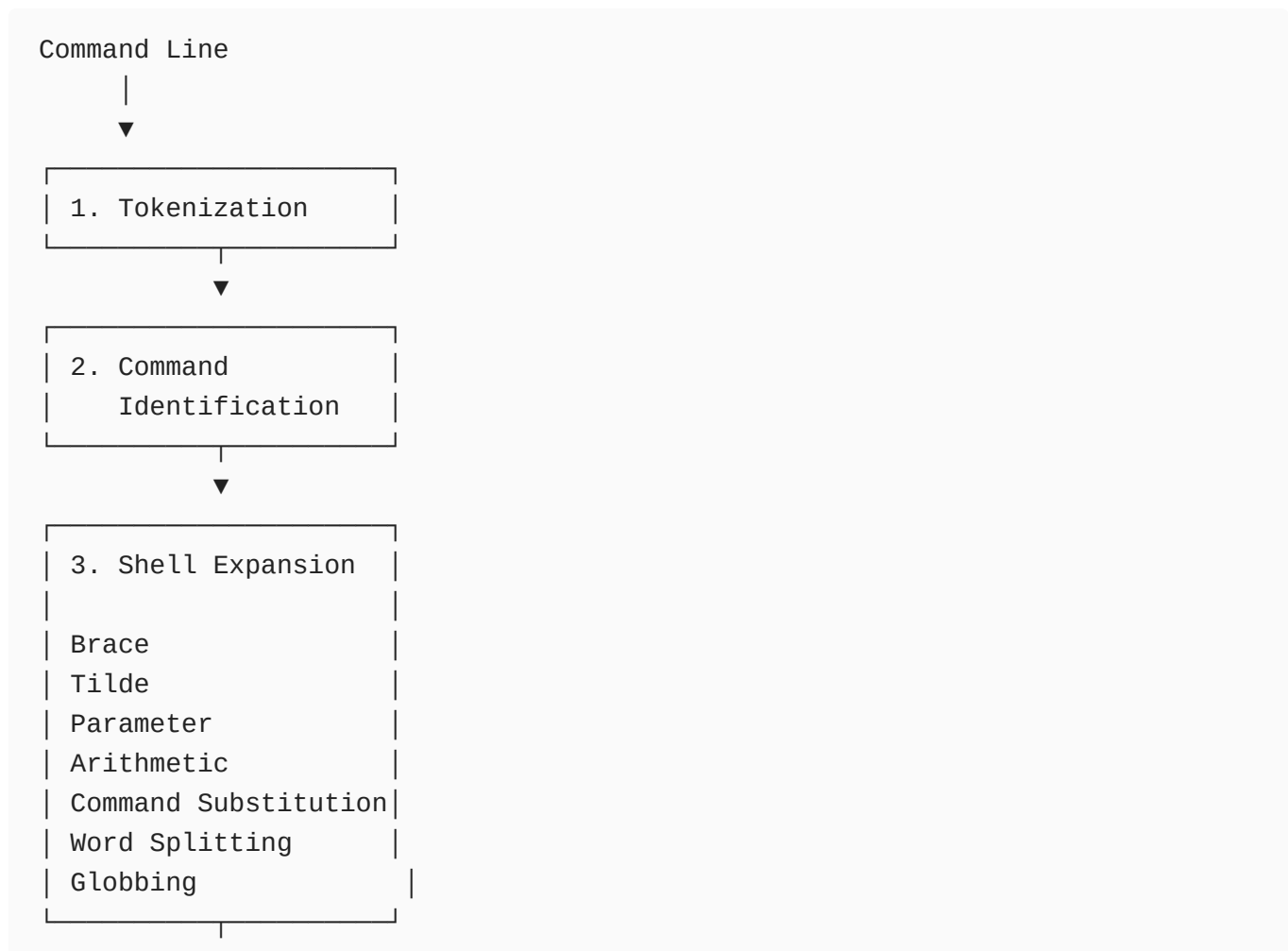
This section is **very important** because it brings together almost everything you've learned so far: quoting, expansions, word splitting, globbing, and redirection.

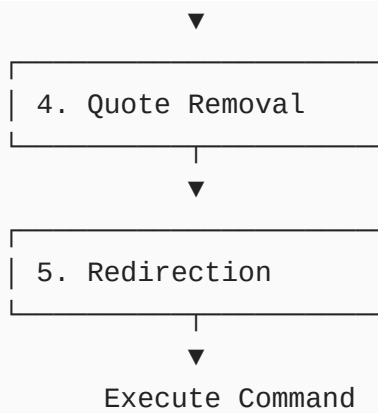
The key idea is:

Bash does not immediately execute the command you type. It first parses and transforms the command through several stages.

1. Big Picture – Bash Command Processing

A useful simplified model is:





Think of this as a **pipeline**.

Each stage prepares the command for the next stage.

2. Quoting

Before understanding the processing stages, you need to understand **quoting**.

What is quoting?

Quoting tells Bash:

"Do not treat this character as having its normal special meaning."

Bash mainly provides three ways to quote characters:

```
\
'...'
"..."
```

3. Backslash \

A backslash generally removes the special meaning of the character immediately following it.

Example:

```
echo \$HOME
```

Output:

```
$HOME
```

Instead of expanding `$HOME` .

Another example:

```
echo hello\ world
```

Output:

```
hello world
```

Without the backslash:

```
echo hello world
```

Bash sees:

```
hello  
world
```

as two separate words.

4. Single Quotes

Single quotes preserve the literal meaning of almost everything inside them.

```
echo '$HOME'
```

Output:

```
$HOME
```

Bash does not perform normal parameter expansion inside single quotes.

Similarly:

```
echo '*.txt'
```

prints:

```
*.txt
```

It does **not** perform globbing.

Mental model

```
'...' → "Treat this as literal text."
```

5. Double Quotes

Double quotes provide controlled interpretation.

Example:

```
name="Sourya"  
echo "$name"
```

Output:

```
Sourya
```

Parameter expansion still occurs.

But double quotes prevent ordinary word splitting and pathname expansion of the expanded result.

For example:

```
name="Sourya Dutta"  
  
printf '<%s>\n' "$name"
```

produces one argument:

```
<Sourya Dutta>
```

Whereas:

```
printf '<%s>\n' $name
```

can undergo word splitting and produce:

```
<Sourya>  
<Dutta>
```

Mental model

```
"..." → "Allow certain expansions, but protect the resulting text from  
normal splitting/globbering."
```

6. Step 1 – Tokenization

Now we get into the actual command-processing pipeline.

What is tokenization?

Tokenization means breaking the command line into meaningful pieces called **tokens**.

These can include:

- words
- operators
- control operators

For example:

```
echo hello | grep hello
```

can be viewed as:

```
echo  
hello
```

```
|  
grep  
hello
```

Here:

```
echo    → word  
hello   → word  
|       → operator  
grep    → word  
hello   → word
```

7. Why Tokenization Is Necessary

Bash needs to understand where one part of the command ends and another begins.

Consider:

```
ls -l /tmp
```

Bash identifies:

```
ls  
-l  
/tmp
```

as separate words.

But:

```
ls -l /tmp | grep txt
```

contains an operator:

```
|
```

So Bash understands that there are two commands connected by a pipe.

8. Important Metacharacters

Bash has characters that have special meaning when they are **unquoted**.

Common examples include:

```
space
tab
newline
|
&
;
(
)
<
>
```

These characters can affect how Bash divides or interprets the command.

Important point

Quoting can remove the special meaning of these characters.

For example:

```
echo hello\ world
```

The space is escaped, so it becomes part of the word.

Similarly:

```
echo "hello world"
```

produces one argument rather than two.

9. Step 2 – Command Identification

After Bash has parsed the command structure, it needs to determine **what kind of command it is dealing with**.

There are two broad concepts to understand:

Simple commands
Compound commands

10. Simple Commands

A simple command generally consists of things such as:

```
command name  
arguments  
variable assignments  
redirections
```

Example:

```
ls -la /home
```

Conceptually:

```
Command → ls  
Argument → -la  
Argument → /home
```

Another example:

```
name=Sourya echo "$name"
```

contains an assignment and a command.

11. Control Operators

Commands can be separated or connected using control operators.

Examples include:

```
;  
&&  
||
```

```
|  
&
```

For example:

```
mkdir test && cd test
```

means:

```
Run mkdir test  
  ↓  
If successful  
  ↓  
Run cd test
```

Whereas:

```
command1 ; command2
```

means:

```
Run command1  
  ↓  
Run command2
```

regardless of whether the first command succeeds, subject to shell error/exit behavior.

12. Compound Commands

A **compound command** contains a larger shell structure.

Common examples include:

```
if  
for  
while  
until  
case
```

For example:

```
if [ -f file.txt ]; then
    echo "File exists"
fi
```

This isn't simply:

```
program + arguments
```

It is a shell language structure.

Similarly:

```
for file in *.log; do
    echo "$file"
done
```

is a compound command.

13. Simple vs Compound Command

Type	Example	Meaning
Simple	<code>ls -l</code>	One basic command
Simple	<code>grep error log.txt</code>	Command + arguments
Compound	<code>if ...; then ...; fi</code>	Conditional structure
Compound	<code>for ...; do ...; done</code>	Loop structure
Compound	<code>while ...; do ...; done</code>	Loop structure
Compound	<code>case ... in ... esac</code>	Pattern-based structure

This distinction becomes very important when you start writing Bash scripts.

14. Step 3 – Shell Expansion

This is probably the most important stage for your current learning.

Bash performs several types of expansion.

A useful order to remember is:

```
Brace Expansion
  ↓
Tilde / Parameter / Arithmetic / Command Substitution
  ↓
Word Splitting
  ↓
Filename Expansion (Globbing)
```

Let's go through them.

15. Brace Expansion

Brace expansion generates multiple words.

Example:

```
echo {red,green,blue}
```

produces:

```
red green blue
```

Another example:

```
echo file{1,2,3}.txt
```

produces:

```
file1.txt file2.txt file3.txt
```

It is important to remember:

Brace expansion generates text; it does not search the filesystem.

16. Tilde Expansion

Tilde expansion handles shortcuts involving home directories.

```
~
```

usually represents your home directory.

For example:

```
cd ~
```

is equivalent in effect to:

```
cd "$HOME"
```

You can also have:

```
~username
```

for another user's home directory when Bash can resolve it.

Bash also supports:

```
~+  
~-
```

where:

```
~+ → current working directory  
~- → previous working directory
```

17. Parameter Expansion

Parameter expansion replaces a variable reference with its value.

Example:

```
name="Sourya"  
echo "$name"
```

Bash replaces:

```
$name
```

with:

```
Sourya
```

You can also use:

```
${name}
```

which is particularly useful when the variable name touches other text.

18. Arithmetic Expansion

Arithmetic expansion performs calculations.

Syntax:

```
$((expression))
```

Example:

```
echo $((10 + 5))
```

Output:

```
15
```

Another:

```
a=10  
b=3
```

```
echo $((a * b))
```

Output:

```
30
```

Bash arithmetic is primarily **integer arithmetic**.

19. Command Substitution

Command substitution allows the output of a command to become part of another command.

Syntax:

```
$(command)
```

Example:

```
current_dir=$(pwd)
```

If `pwd` produces:

```
/home/sourya
```

then:

```
current_dir
```

contains that output.

Another example:

```
echo "Hostname: $(hostname)"
```

The output of `hostname` becomes part of the argument passed to `echo`.

20. Word Splitting

After certain expansions, Bash can split unquoted results into multiple words.

Consider:

```
name="Sourya Dutta"
```

Then:

```
printf '<%s>\n' $name
```

can become:

```
printf '<%s>\n' Sourya Dutta
```

giving:

```
<Sourya>  
<Dutta>
```

But:

```
printf '<%s>\n' "$name"
```

gives:

```
<Sourya Dutta>
```

because the expansion is quoted.

21. IFS

Word splitting is controlled by:

```
IFS
```

which means:

Internal Field Separator

The default IFS contains:

```
space
tab
newline
```

For example:

```
data="one two three"
```

Unquoted:

```
printf '<%s>\n' $data
```

can split the value into:

```
<one>
<two>
<three>
```

22. Custom IFS

You can change the separator.

For example:

```
IFS=", "
data="apple,banana,orange"
```

Then an unquoted expansion can be split using commas.

This becomes useful when processing:

- CSV-like data
- command output
- lists

- configuration values

However, in modern Bash scripting, `read`, arrays, and carefully controlled parsing are often safer than relying heavily on global IFS changes.

23. Globbing

After word splitting, Bash can perform filename expansion.

The main patterns are:

```
*      → zero or more characters
?      → exactly one character
[]     → one character from a set/range
```

Example:

```
ls *.log
```

If the directory contains:

```
auth.log
system.log
error.log
```

Bash can turn:

```
*.log
```

into:

```
auth.log system.log error.log
```

before executing `ls`.

24. Important Difference: Word Splitting vs Globbing

These are often confused.

Word splitting

Takes a value like:

```
one two three
```

and can produce:

```
one  
two  
three
```

Globbing

Takes a pattern like:

```
*.log
```

and matches filesystem names:

```
auth.log  
system.log  
error.log
```

So:

```
Word splitting → separates text into words
```

```
Globbing → matches filenames
```

25. Step 4 – Quote Removal

Once the relevant expansions have been performed, Bash removes the syntactic quoting characters that served to control interpretation.

For example:

```
echo "Hello"
```

The quotes tell Bash:

| Treat `Hello` as one quoted word.

They aren't normally passed to `echo` as literal quote characters.

Similarly:

```
echo \ $HOME
```

uses `\` to prevent expansion, and the quoting/escaping syntax is handled by the shell.

Core idea

| **Quotes are instructions to the shell, not normally part of the final argument.**

26. Step 5 – Redirection

The final major stage in this lesson is **redirection**.

Redirection determines where a command's standard streams come from or go to.

Linux processes normally have:

```
STDIN
STDOUT
STDERR
```

with file descriptors:

```
0
1
2
```

27. STDIN

```
STDIN = 0
```

Standard input.

Normally:

```
Keyboard → Program
```

You can redirect it from a file:

```
cat < input.txt
```

Now:

```
input.txt → STDIN → cat
```

28. STDOUT

```
STDOUT = 1
```

Standard output.

Normally:

```
Program → Terminal
```

Redirect it:

```
ls > files.txt
```

Now:

```
Program → STDOUT → files.txt
```

> normally **overwrites** the target file.

29. Append with >>

```
ls >> files.txt
```

means:

Send STDOUT to `files.txt`, appending instead of replacing its existing contents.

Remember:

```
>   → overwrite
>>  → append
```

30. STDERR

```
STDERR = 2
```

Standard error.

For example:

```
ls /does-not-exist
```

produces an error through STDERR.

You can redirect it:

```
ls /does-not-exist 2> errors.txt
```

Now:

```
STDERR → errors.txt
```

31. STDOUT and STDERR Separately

You can separate normal output from errors:

```
command > output.txt 2> errors.txt
```

Result:

```
graph LR; STDOUT["STDOUT (1)"] --- Branch; Branch --- output["output.txt"]; Branch --- STDERR["STDERR (2)"]; STDERR --- errors["errors.txt"]
```

This is extremely useful in automation.

32. Redirect Both

A common Bash form is:

```
command > output.txt 2>&1
```

This means:

```
STDOUT → output.txt
STDERR → same destination as STDOUT
```

Therefore both go into:

```
output.txt
```

Bash also supports:

```
command &> output.txt
```

for redirecting both standard output and standard error.

33. Practical Example – Following the Entire Pipeline

Consider:

```
name="Sourya"  
echo "Hello $name" > greeting.txt
```

Let's conceptually walk through it.

Stage 1 – Tokenization

Bash identifies pieces such as:

```
name="Sourya"  
echo  
"Hello $name"  
>  
greeting.txt
```

The exact parser representation is more nuanced, but this is a useful learning model.

Stage 2 – Command Identification

Bash recognizes the command structure.

There is:

```
Variable assignment
```

and a simple command:

```
echo
```

with an argument and redirection.

Stage 3 – Expansion

The parameter:

```
$name
```

is expanded.

So:

```
"Hello $name"
```

becomes conceptually:

```
"Hello Sourya"
```

Because it is inside double quotes, it remains one word.

Stage 4 – Quote Removal

The double quotes are removed as syntactic quoting.

The argument effectively becomes:

```
Hello Sourya
```

as one argument.

Stage 5 – Redirection

The:

```
>
```

redirects STDOUT to:

```
greeting.txt
```

So instead of:

```
echo → Terminal
```

we have:

```
echo → STDOUT → greeting.txt
```

The file contains:

```
Hello Sourya
```

34. Example – Why Processing Order Matters

Suppose:

```
name="Sourya Dutta"
```

and:

```
echo "$name"
```

Because the expansion is quoted:

```
$name  
↓  
Sourya Dutta  
↓  
remains one word
```

But:

```
echo $name
```

can result in:

```
$name  
↓  
Sourya Dutta  
↓
```

```
word splitting
```

```
↓
```

```
Sourya
```

```
Dutta
```

Understanding the processing order lets you explain **why** the commands behave differently instead of simply memorizing "always use quotes."

35. The IFS + Tilde Expansion Example

A common beginner mistake is expecting every occurrence of `~` to perform tilde expansion.

For example, conceptually:

```
path("~/file.txt")
```

does **not** perform tilde expansion just because the value contains `~`.

Why?

Because tilde expansion happens when Bash parses a suitable **literal tilde at the beginning of a word**, not simply because the characters `~` appear inside a variable's value.

So:

```
path("~/file.txt")
echo "$path"
```

does not magically turn it into:

```
/home/sourya/file.txt
```

36. Correct Ways to Handle Home Paths

Instead of relying on tilde expansion inside variable contents, use:

```
path="$HOME/file.txt"
```

This is clearer and more reliable.

Or use literal tilde directly in a command where tilde expansion applies:

```
cat ~/file.txt
```

Important distinction

```
~/file.txt
```

→ tilde expansion can happen.

But:

```
path="~/file.txt"
```

→ the `~` is just part of the variable's string.

Then:

```
echo "$path"
```

doesn't trigger a second tilde-expansion pass.
